

Salesforce Platform Developer – Project Documentation

HandsMen Threads

A Custom CRM for a Growing Men's Fashion Brand

Project Title	HandsMen Threads – Salesforce CRM Implementation
Platform	Salesforce Developer Edition
Document Type	Project Documentation
Prepared By	Vishal R
Submission Date	September 2026

Project Overview

HandsMen Threads is a men's fashion label that had reached the point where spreadsheets were no longer keeping up with the business. Customer details sat in one file, the product catalogue in another, and orders in a third, which meant figures had to be reconciled by hand and nobody had a dependable view of what stock was actually left on the shelf. This project replaces that arrangement with a purpose-built CRM on the Salesforce platform, where customers, products, orders, inventory and marketing activity are all held as related records in a single org.

The solution leans on declarative tooling wherever it is sufficient — custom objects, validation rules and Flow Builder — and drops into Apex only for the handful of calculations that clicks cannot express cleanly, namely order totalling, live stock adjustment and loyalty tier recalculation. Everything is surfaced through one Lightning application, so the Customer, Order, Product, Inventory, Campaign and Reports tabs open from a single place rather than being scattered across the org.

Objectives

The overarching aim was to lift the repetitive, error-prone parts of running the brand off a person's to-do list and, at the same time, give the sales, inventory and marketing teams a shared and trustworthy picture of the business. In business terms, that translates into fewer order-entry mistakes, faster customer communication, and stock decisions based on current numbers rather than on the last time somebody updated a sheet. The specific goals set at the outset were:

- Automate the recurring administrative work — order confirmation emails, loyalty tier movement and low-stock warnings — so none of it depends on somebody remembering.
- Prevent poor-quality data from entering the system at the point of save, chiefly through validation on customer email addresses and product stock levels.
- Give each team self-service visibility of the orders, stock positions and customer activity that matter to them, without having to ask a colleague.
- Keep every department working from one shared set of records instead of maintaining private copies.
- Make the customer experience feel more considered through a tiered loyalty scheme and prompt automated updates.

Phase 1: Requirement Analysis and Planning

Understanding the Business Requirements

Before any configuration was attempted, the existing manual process was examined to work out precisely where it was failing. Three problems stood out. Order values were being calculated by hand, so arithmetic slips went unnoticed until a customer queried an invoice. There was no reliable signal that a product was running low, which meant restocking happened reactively. And customers received nothing at all once an order had been placed, leaving the team to field "has it gone through?" enquiries. What the business needed was a single home for customer and product information, order totals derived automatically, a hard stop on stock falling below zero, and customers kept informed without additional effort from staff.

Defining the Project Scope and Objectives

Rather than attempting to model the entire business at once, the scope was deliberately narrowed to the pieces that would deliver the most relief soonest:

- A data model built from custom objects for Customers, Products, Orders, Inventory and Marketing Campaigns.
- Automation covering the three most painful manual tasks: order confirmation, stock deduction and loyalty status maintenance.
- Validation on customer email addresses and product stock so that invalid records cannot be committed.
- Distinct profiles and roles for the Sales and Inventory functions, as their day-to-day needs differ considerably.
- A single Lightning application binding the components together rather than a loose collection of tabs.

Data Model and Security Model

The data model rests on four custom objects, supported by an Inventory object that tracks physical stock separately from the product definition itself:

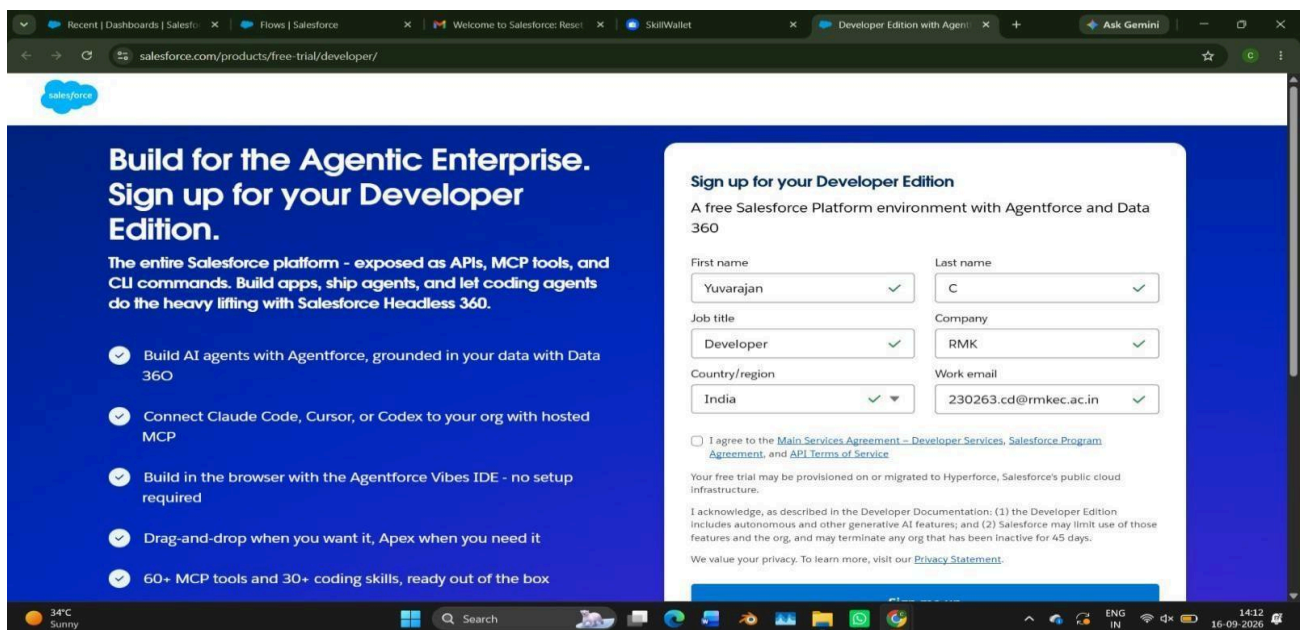
- HandsMen Customer (Customer_c) — stores Name, Email, Phone, Address and Loyalty_Status_c.
- HandsMen Product (Product_c) — stores Name, Description, Unit_Price_c and Stock_c.
- HandsMen Order (Order_c) — stores the order date, Quantity_c and Total_Amount_c, and relates back to both a Customer and a Product.
- Marketing Campaign (Marketing_Campaign_c) — records promotional and marketing activity.
- Inventory — holds stock quantity by product and warehouse and is the record that drives the low-stock alerts.

Security is handled in two complementary layers. Profiles govern capability — which objects, fields and pages a user can reach at all — while the role hierarchy governs breadth of visibility within whatever the profile permits. The Sales Manager role sits above Sales Representative, for example, so a manager sees the orders raised across the team while an individual representative sees only their own.

Phase 2: Salesforce Development – Backend and Configuration

Setting Up the Environment

A free Salesforce Developer Edition org was provisioned from developer.salesforce.com/signup, and all build work, testing and screen capture for this project was carried out inside that org.

The screenshot shows the Salesforce Developer Edition sign-up page. On the left, there's a blue sidebar with the heading "Build for the Agentic Enterprise. Sign up for your Developer Edition." Below this, it says "The entire Salesforce platform - exposed as APIs, MCP tools, and CLI commands. Build apps, ship agents, and let coding agents do the heavy lifting with Salesforce Headless 360." There are five bullet points: "Build AI agents with Agentforce, grounded in your data with Data 360", "Connect Claude Code, Cursor, or Codex to your org with hosted MCP", "Build in the browser with the Agentforce Vibes IDE - no setup required", "Drag-and-drop when you want it, Apex when you need it", and "60+ MCP tools and 30+ coding skills, ready out of the box". On the right, there's a white form titled "Sign up for your Developer Edition" with the subtitle "A free Salesforce Platform environment with Agentforce and Data 360". The form has fields for First name (Yuvarajan), Last name (C), Job title (Developer), Company (RMK), Country/region (India), and Work email (230263.cd@rmkec.ac.in). There's a checkbox for "I agree to the Main Services Agreement - Developer Services, Salesforce Program Agreement, and API Terms of Service". Below the checkbox, it says "Your free trial may be provisioned on or migrated to Hyperforce, Salesforce's public cloud infrastructure." and "I acknowledge, as described in the Developer Documentation: (1) the Developer Edition includes autonomous and other generative AI features; and (2) Salesforce may limit use of those features and the org, and may terminate any org that has been inactive for 45 days." At the bottom, it says "We value your privacy. To learn more, visit our Privacy Statement." The browser's address bar shows "salesforce.com/products/free-trial/developer/". The Windows taskbar at the bottom shows the date and time as 14:12 on 16-09-2024.

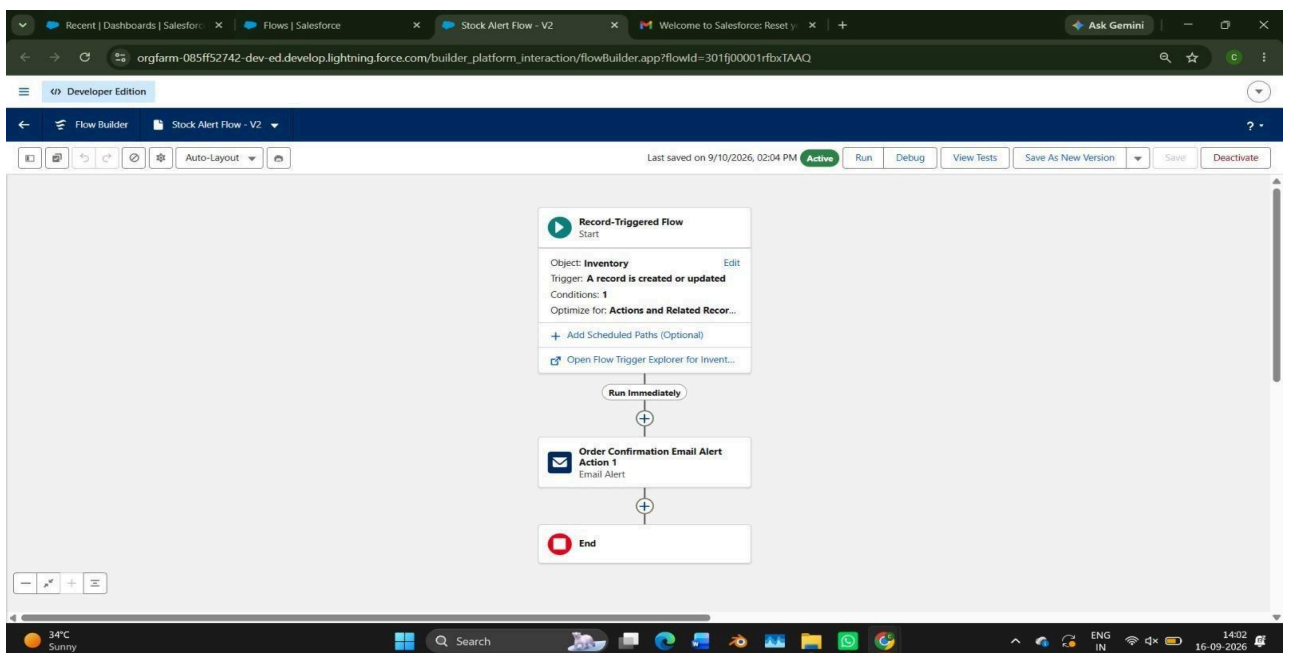
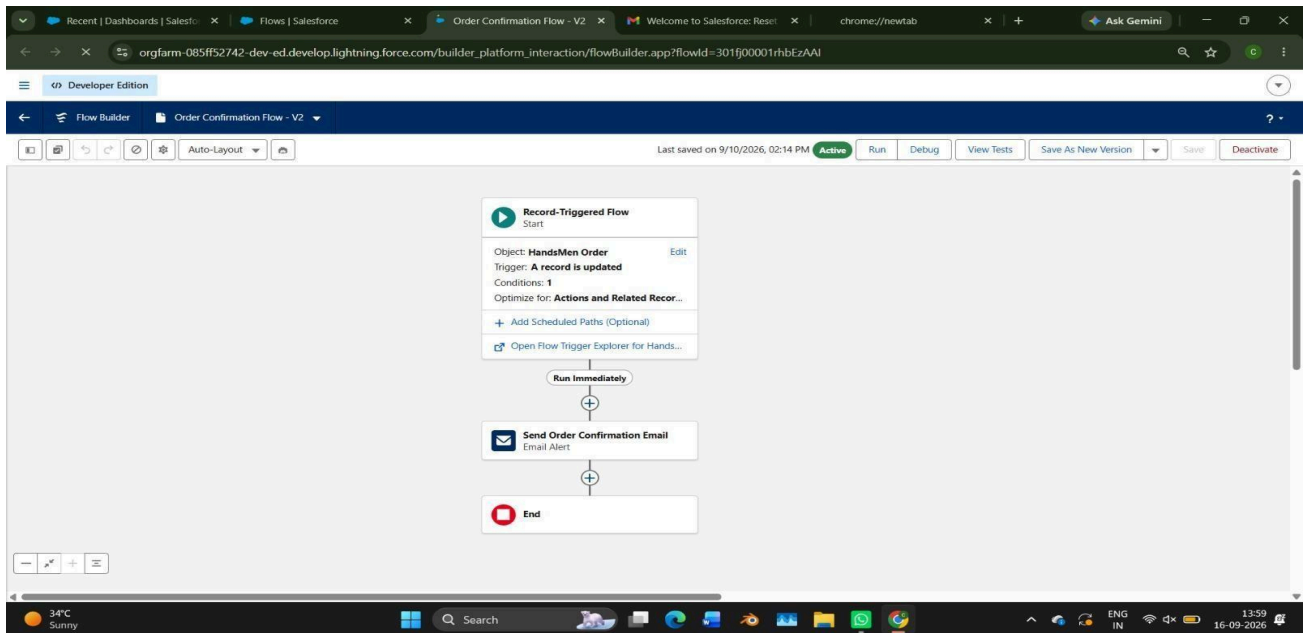
Objects, Fields, Validation Rules and Automation

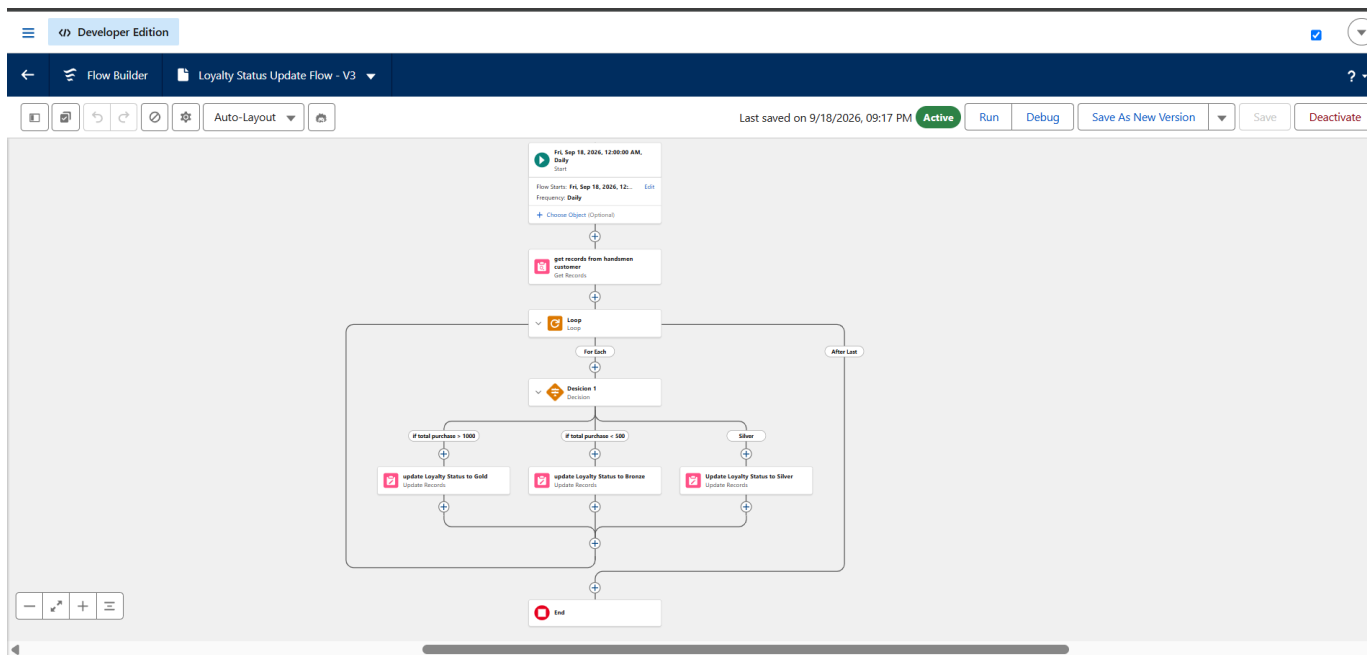
The four custom objects defined in Phase 1 (Customer_c, Product_c, Order_c and Marketing_Campaign_c) were created first, each with its own custom tab so that records can be entered and browsed from within the application rather than through Setup.

Two validation rules carry the data-quality load. The first sits on the product record and refuses any save that would leave Stock_c below zero; this is the safeguard that makes overselling impossible. The second sits on the customer record and requires the email field to contain "@gmail.com" before the record will commit, which keeps contact details in a predictable format for the automated mailings.

Automation was built in Flow Builder in preference to Apex wherever the logic allowed it, on the grounds that a flow is far easier for a future administrator to read and amend. Three flows were created:

- An order confirmation flow, triggered on the creation of a HandsMen Order record, which emails the customer as soon as the order is logged.
- A low-stock alert flow, triggered on the Inventory object, which notifies the inventory manager once a product's quantity falls past the threshold.
- A schedule-triggered flow running once daily that iterates over the customer base, evaluates cumulative purchases and reassigns each customer to the appropriate loyalty tier.





Apex Classes and Triggers

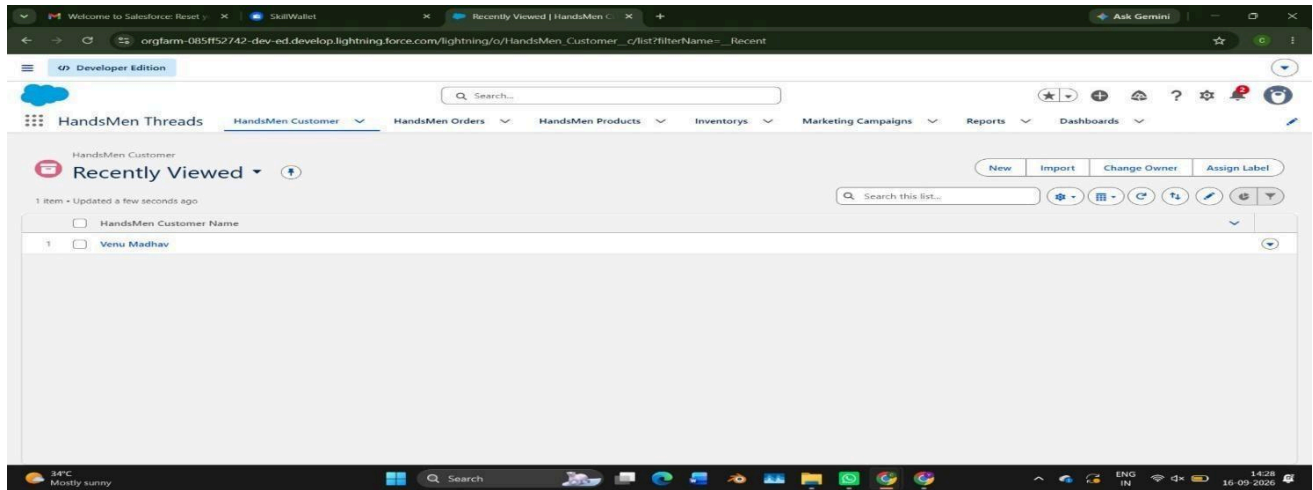
A small amount of logic proved awkward to express declaratively, so three Apex triggers were written to cover it. Each performs a short, synchronous calculation at the moment a record is saved, so no batch, queueable or future methods were required for this build.

- OrderTrigger derives Total_Amount_c from the ordered quantity and the related product's unit price, removing the manual calculation entirely.
- ProductStockUpdateTrigger subtracts the ordered quantity from the product's stock once the order is confirmed, relying on the stock validation rule as a backstop against a negative balance.
- CustomerLoyaltyUpdateTrigger re-evaluates Loyalty_Status_c the instant a customer's purchase total changes, so the tier is accurate immediately rather than only after the nightly scheduled flow has run.

Phase 3: UI/UX Development and Customisation

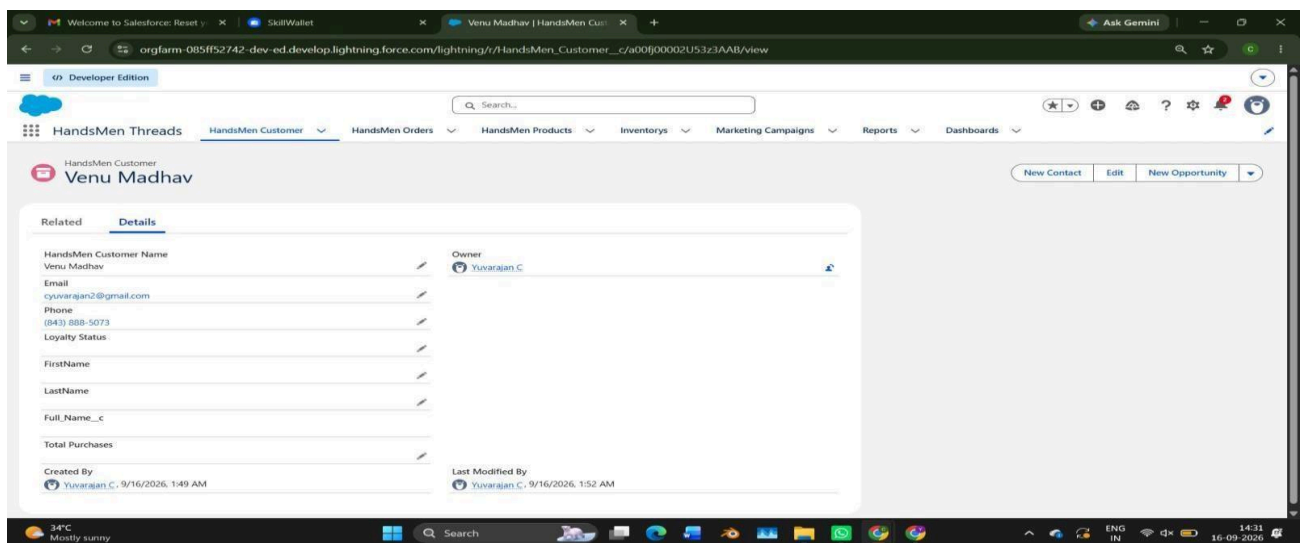
Building the Application in App Manager

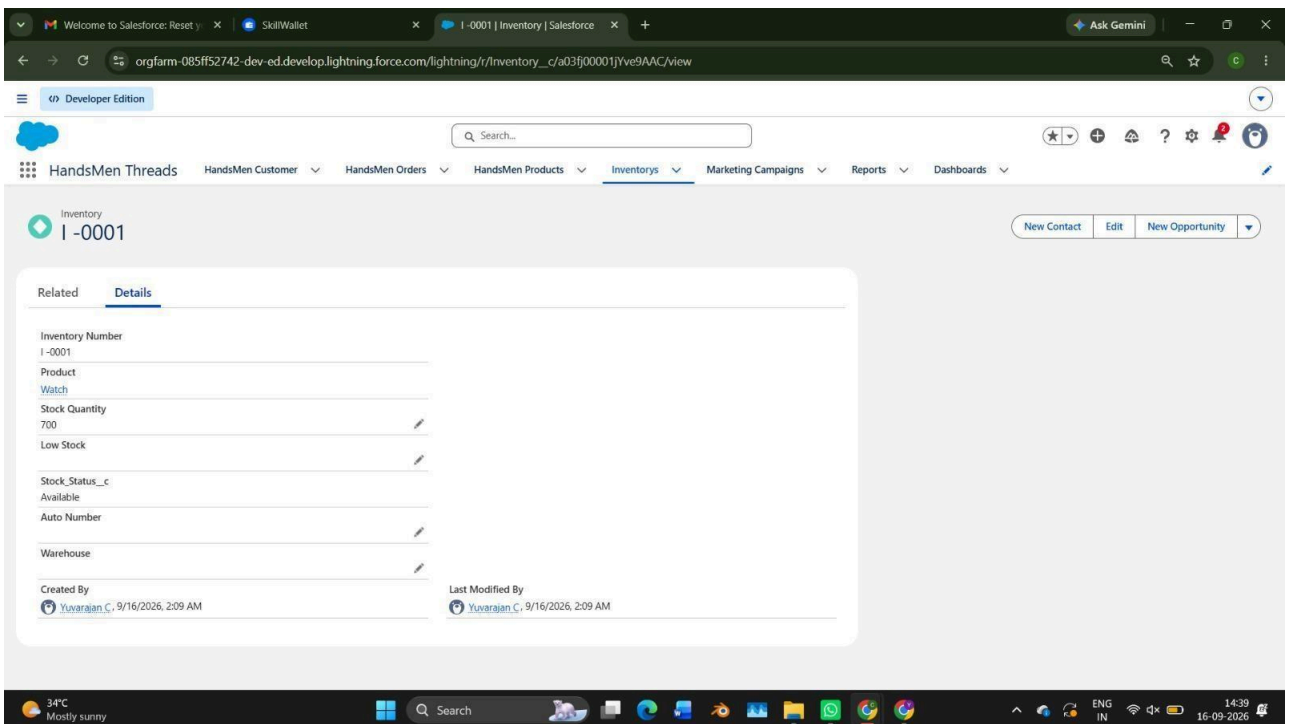
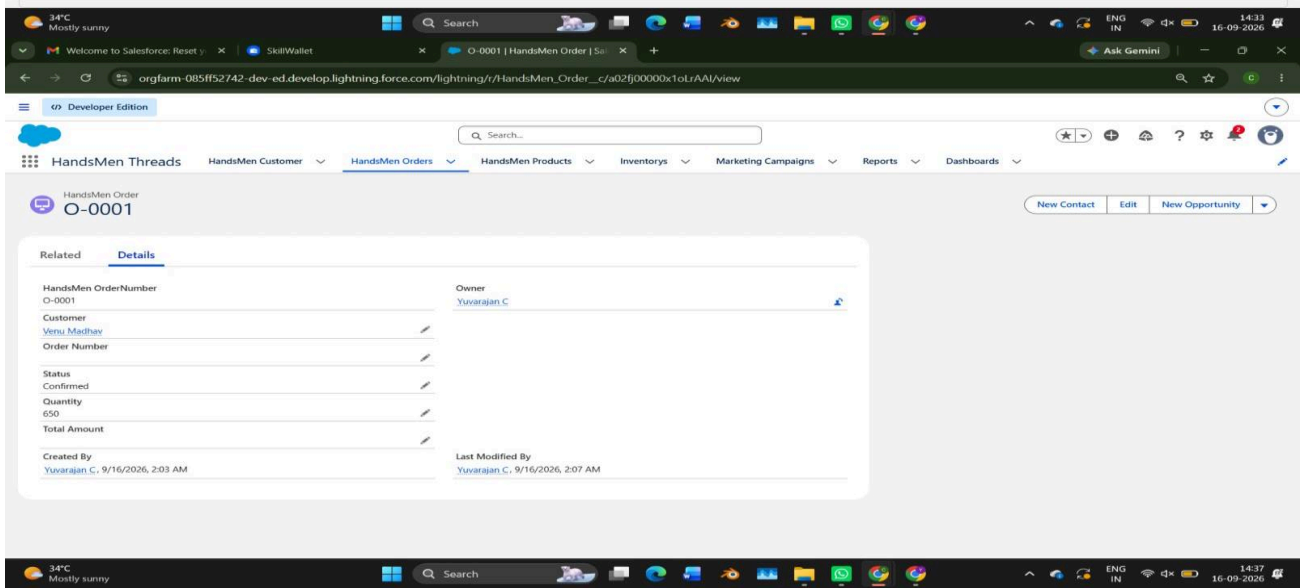
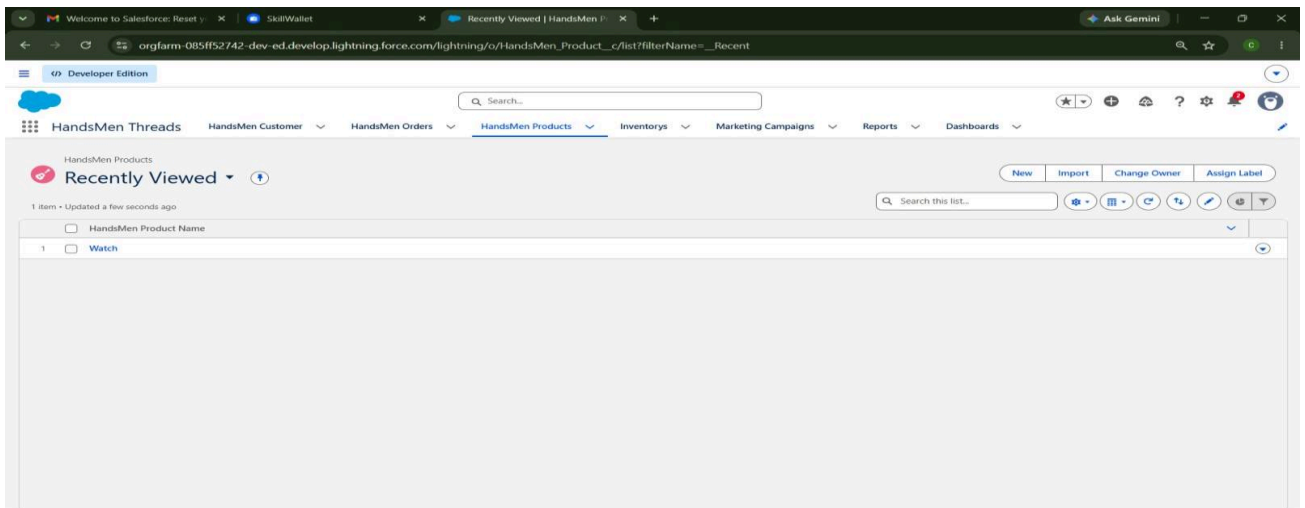
A Lightning application named "HandsMen Threads" was assembled through App Manager, with the Customer, Orders, Products, Inventory, Reports and Dashboards tabs added to its navigation. The team therefore opens one application rather than locating each tab separately through the App Launcher.



Page Layouts and Record Creation

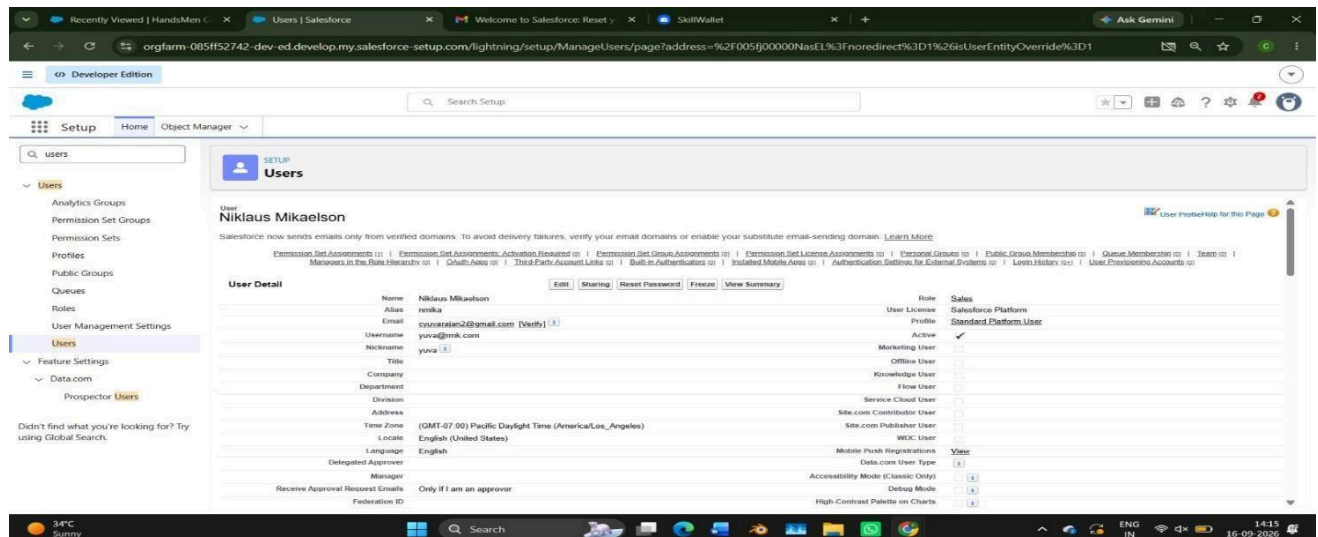
Page layouts were kept close to standard for each object; the work here was mainly a matter of ordering fields so that the details consulted most often — name, contact information, stock on hand and order total — appear above the fold without scrolling. The Customer, Product, Order and Inventory record pages each follow this pattern, with related lists placed on the Related tab so the detail view stays uncluttered.





User Management

To confirm that the multi-user design behaved as intended, two test users were created and assigned to different profiles and roles — one configured for Sales and one for Inventory — so that each sees only the records and functions relevant to their job.



Reports and Dashboards

The Reports and Dashboards tabs are wired into the application and provide the foundation for sales and stock analytics. Detailed dashboards have not yet been authored; this is recorded among the future enhancements at the close of this document.

Phase 4: Data Migration, Testing and Security

Data Loading

Because the volume involved in this build was small, sample Customer, Product, Order and Inventory records were keyed in directly through the application interface. Were the solution rolled out against the brand's live customer and product lists, the Data Import Wizard would suit a modest guided import of standard and custom objects, while Data Loader would be the better fit for a larger CSV-based migration where field mapping and batch handling matter.

Field History, Duplicate and Matching Rules

Loyalty_Status_c and Stock_c are the two fields whose history is most worth preserving, so Field History Tracking on them records who changed a value, from what, and when — which is exactly what is needed if a tier or a stock figure later looks wrong. On the Customer object, duplicate and matching rules keyed on email and phone number prevent the same individual from being entered twice.

Profiles, Roles, Permission Sets and Sharing

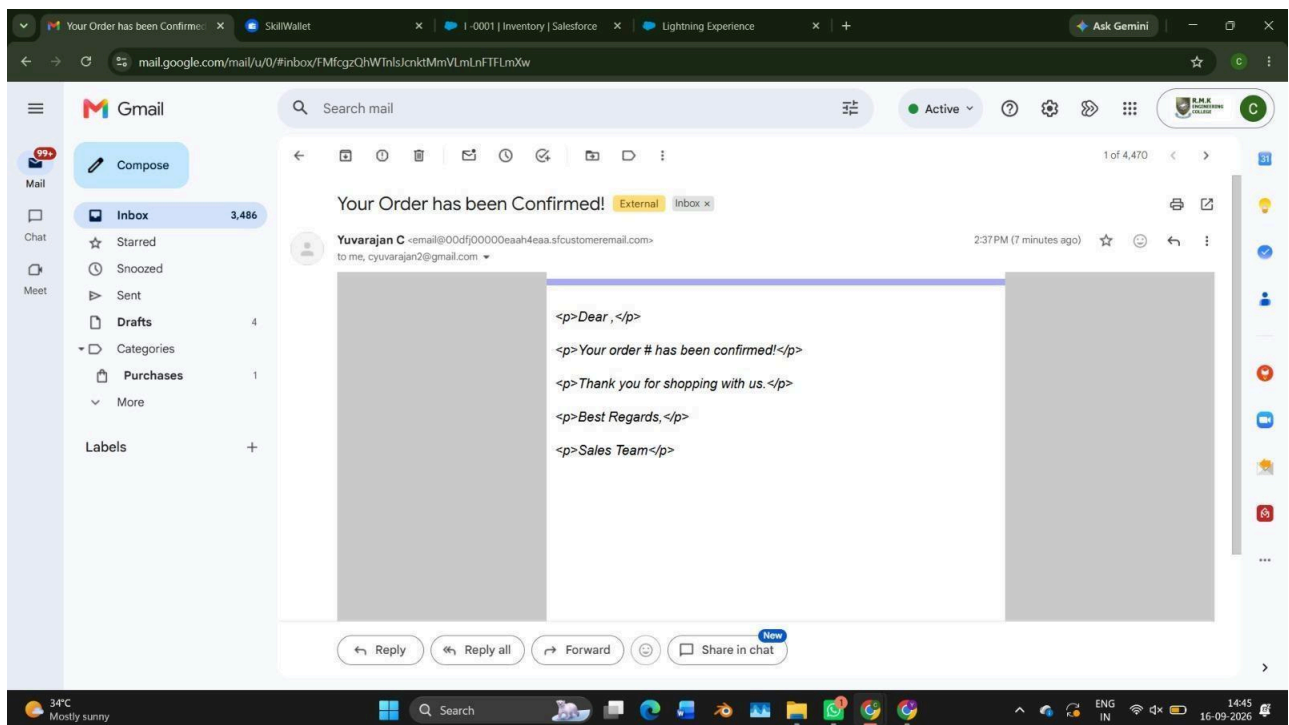
The Standard User profile was cloned to produce a "HandsMen Sales Profile" trimmed to what a sales representative genuinely needs, in preference to granting everyone the same broad permission set. Roles such as Sales Manager and Inventory Manager are arranged in a hierarchy so that a manager inherits visibility of the records owned beneath them with no extra sharing configuration. Where an individual needs slightly more access than their profile grants, a permission set is used so the profile is not loosened for everybody else. Sharing rules are deliberately sparse at this stage, as the org-wide defaults cover present requirements, but they would be the mechanism of choice if, for instance, Inventory later needed sight of Order records outside their own.

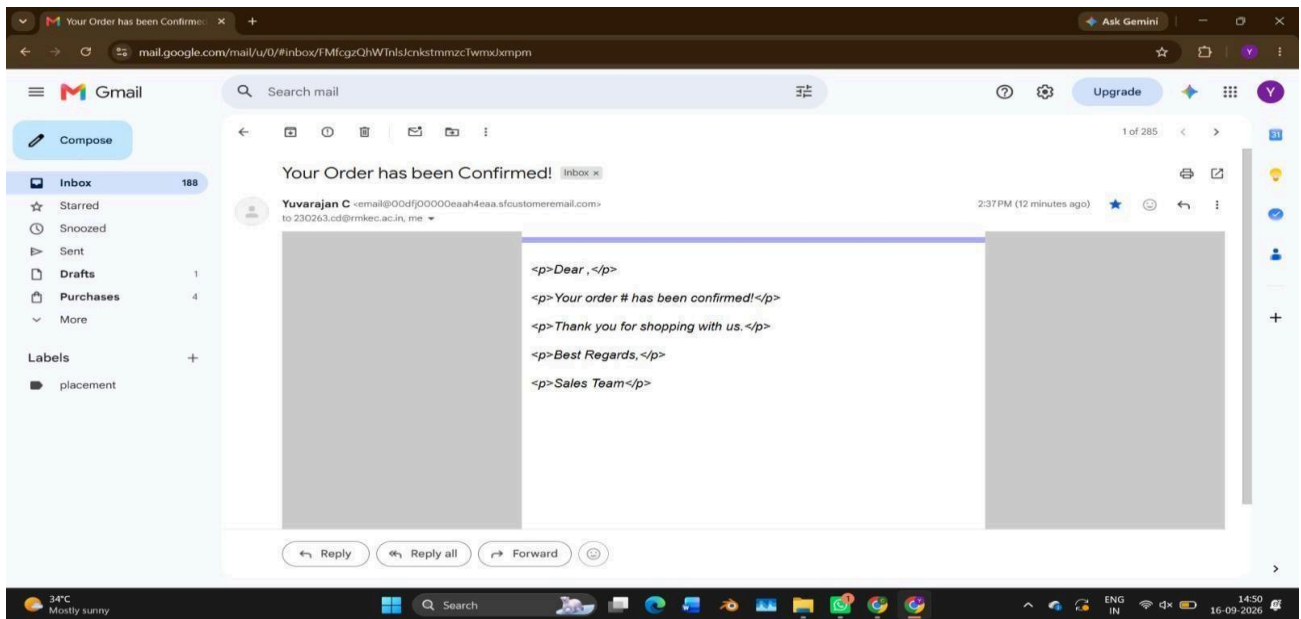
Testing Approach and Test Classes

Each of the three Apex triggers has an accompanying test class. Coverage was written for both the positive path — an order saves cleanly and the total, stock level and loyalty status all settle on the correct values — and the negative path, where an attempt to oversell stock or to save a customer with a malformed email address is expected to fail. Testing the negative case matters as much as the positive one, since it demonstrates that the validation rules genuinely block bad data rather than merely appearing to.

Alongside the automated tests, the scenarios in the table below were executed manually within the org to confirm that each piece of automation behaves correctly from end to end. In every case the observed outcome matched the expected result recorded here.

Feature	Test Scenario	Expected Result
Customer email validation rule	Save a customer whose email does not end in "@gmail.com"	The record is rejected and an inline error is displayed
Order total trigger	Raise an order for one unit of a product priced at \$500	Total_Amount_c is populated as \$500 without manual entry
Order confirmation flow	Save a newly confirmed order record	A confirmation email reaches the customer on the linked record
Stock deduction trigger	Move an order into Confirmed status	Stock_c on the related product falls by the ordered quantity
Product stock validation rule	Attempt an order larger than the quantity in hand	The save is stopped, so the stock figure never turns negative
Low stock alert flow	Reduce an Inventory record below the alert threshold	The Inventory Manager receives the low-stock notification email
Loyalty status trigger and scheduled flow	Push a customer's cumulative purchases past a tier boundary	Loyalty_Status_c moves to the matching tier, e.g. Bronze





Phase 5: Deployment, Documentation and Maintenance

Deployment Strategy

The whole solution was constructed and tested inside a single Developer Edition org, so no cross-environment deployment was needed for this exercise. Promoting it to a sandbox and then to production would be done with change sets, released in dependency order: custom objects and fields first, followed by validation rules and flows, and finally the Apex triggers together with their test classes so that the required code coverage travels with them. Each change set would be validated against the target org before an actual deployment is run.

Maintenance and Monitoring

Ongoing upkeep is largely a matter of periodically reviewing flow run history to confirm that the order confirmation, low-stock alert and loyalty flows are still firing as expected, and revisiting the validation rules whenever a business rule shifts — for example, if customers are eventually permitted to register with addresses other than Gmail. Field history on the key fields provides the audit trail needed to investigate anything that looks out of place.

Troubleshooting Approach

When something does not behave as expected, the first place to look is the run history of the flow concerned, or the Apex debug logs where a trigger is involved, simply to establish where execution actually stopped. It is also worth confirming that a validation rule is not quietly rejecting the save before concluding that the automation itself is at fault. One practical lesson from testing: the low-stock alert occasionally landed in the recipient's spam folder, which turned out to be an email deliverability setting rather than a defect in the flow — worth bearing in mind if alerts appear to go missing.

Conclusion

This project moved HandsMen Threads' customer, order, inventory and campaign data out of spreadsheets and into a single Salesforce org, with the laborious parts of the process — totalling orders, deducting stock, issuing confirmations and maintaining loyalty tiers — now handled automatically rather than by hand. The build combines declarative configuration with a modest amount of Apex, is protected by validation rules at the point of entry, and is covered by both automated test classes and a documented set of manual test cases. There is clearly further ground to cover, but even in its current form the system removes a meaningful amount of routine effort and offers a far clearer view of sales and stock than the previous arrangement could.

Future Enhancements

Several extensions are worth pursuing beyond the present scope:

- A self-service customer portal allowing customers to review past orders and loyalty points themselves instead of contacting the team.
- A mobile application built on the Salesforce Mobile SDK, aimed primarily at in-store staff who need to check stock or raise an order away from a desktop.
- Fully built sales and inventory dashboards, since the Reports and Dashboards tabs are in place but not yet populated with meaningful analytics.
- Einstein-driven product recommendations that suggest items based on a customer's purchase history.
- WhatsApp or SMS integration for order and dispatch updates, reaching customers who do not routinely check email.
- A chatbot capable of answering common customer questions without a member of staff being involved in every exchange.